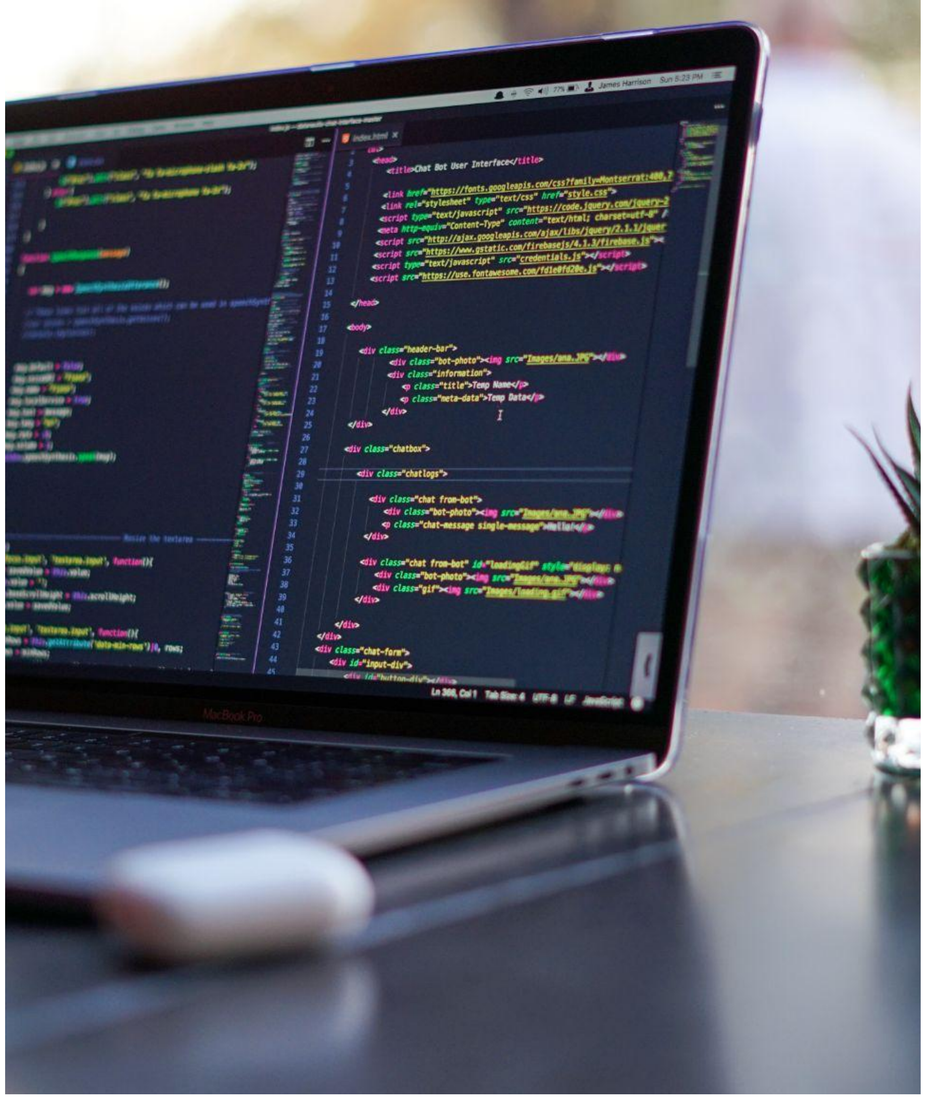
1.
OVERVIEW

An overview of Requirement 6 (“Develop Securely”), and its sub-requirements.



REQ. 6: DEVELOP SECURELY **OVERVIEW**

Requirement 6 (Develop and Maintain Secure Systems and Software) is about securing both developed and vendor software.

- Ours is protected in coding, and vendors' in patching;
- It's one of the most elusive requirements - in your own SDLC, devs care about functionality and not security.

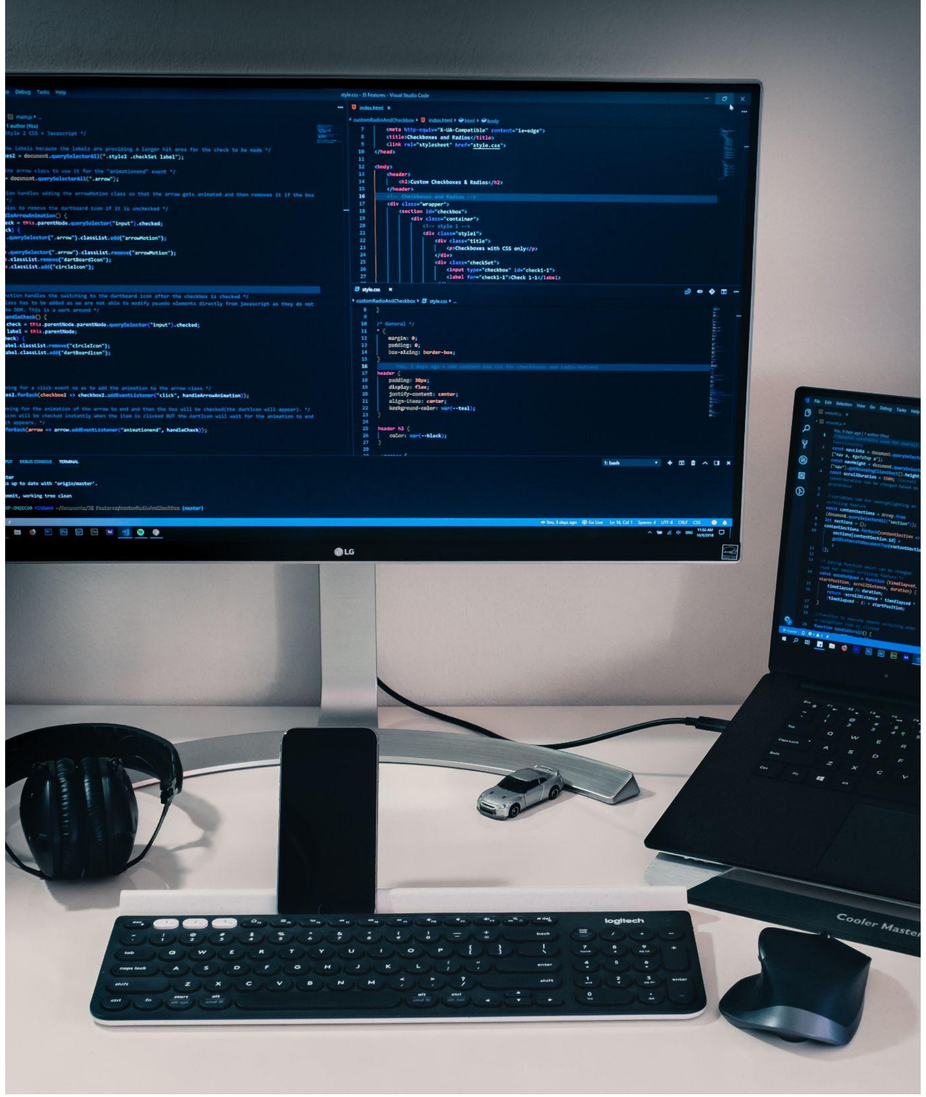
This requirement focuses mostly on:

- Having good SDLC (SW dev. lifecycle) practices and secure coding techniques, (e.g., protections from code injections):
- Installing vendor software patches to prevent vulnerabilities being exploited;
- Securing codebases and configurations for code;

REQ. 6: DEVELOP SECURELY **OVERVIEW**

The sub-requirements for this requirement include:

- 6.1 Processes and mechanisms for developing and maintaining secure systems and software are defined and understood.
- 6.2 Bespoke and custom software are developed securely.
- 6.3 Security vulnerabilities are identified and addressed.
- 6.4 Public-facing web applications are protected against attacks.
- 6.5 Changes to all system components are managed securely.



REQ. 6: DEVELOP SECURELY **OVERVIEW**

6.1 Processes and mechanisms for developing and maintaining secure systems and software are defined and understood

- Our usual documentation sub-requirement. Documentation must exist, be accurate, updated, and in use for all operations regarding secure development, and the same for role and responsibility documentation;

This is usually tested through:

- Verifying documents and interviewing personnel regarding the documentation of these processes and policies, and also verifying that roles and responsibilities are accurate and in use;

REQ. 6: DEVELOP SECURELY **OVERVIEW**

6.2 Bespoke and custom software are developed securely

- Basically, your developers must have good security practices when coding (not as easy as it seems. Everyone wants to implement features... not protect inputs);
- Regardless of method (agile, waterfall...);
- They must be trained in terms of security, they must protect against attacks such as injection attacks, buffer overflows, API manipulation, XSS and CSRF attacks, etc;

This is usually tested through:

- Examining the software documentation to ensure security risks are identified - and tackled. As well as up-to-date;
- Examining policies and procedures for developing the SW;
- Examining training records and interviewing personnel;

REQ. 6: DEVELOP SECURELY **OVERVIEW**

6.3 Security vulnerabilities are identified and addressed

- For both in-house and purchased software, vulnerabilities must be regularly identified and ranked. Vendors usually publish these, but other sources (e.g., InfoSec forums) exist;
- For in-house software, patches are internally developed;
- Critical patches < 1 month. Others at an “appropriate” pace;
- This is not the same as vulnerability scanning in Req. 11!

This is usually tested through:

- Examining documentation on the organization’s list of SW vulnerabilities, including risk ranking and update dates;
- Verifying the inventory of SW apps, as well as their respective vulnerabilities. Also, verifying SW versions and comparing to latest patches, and update logs, too;

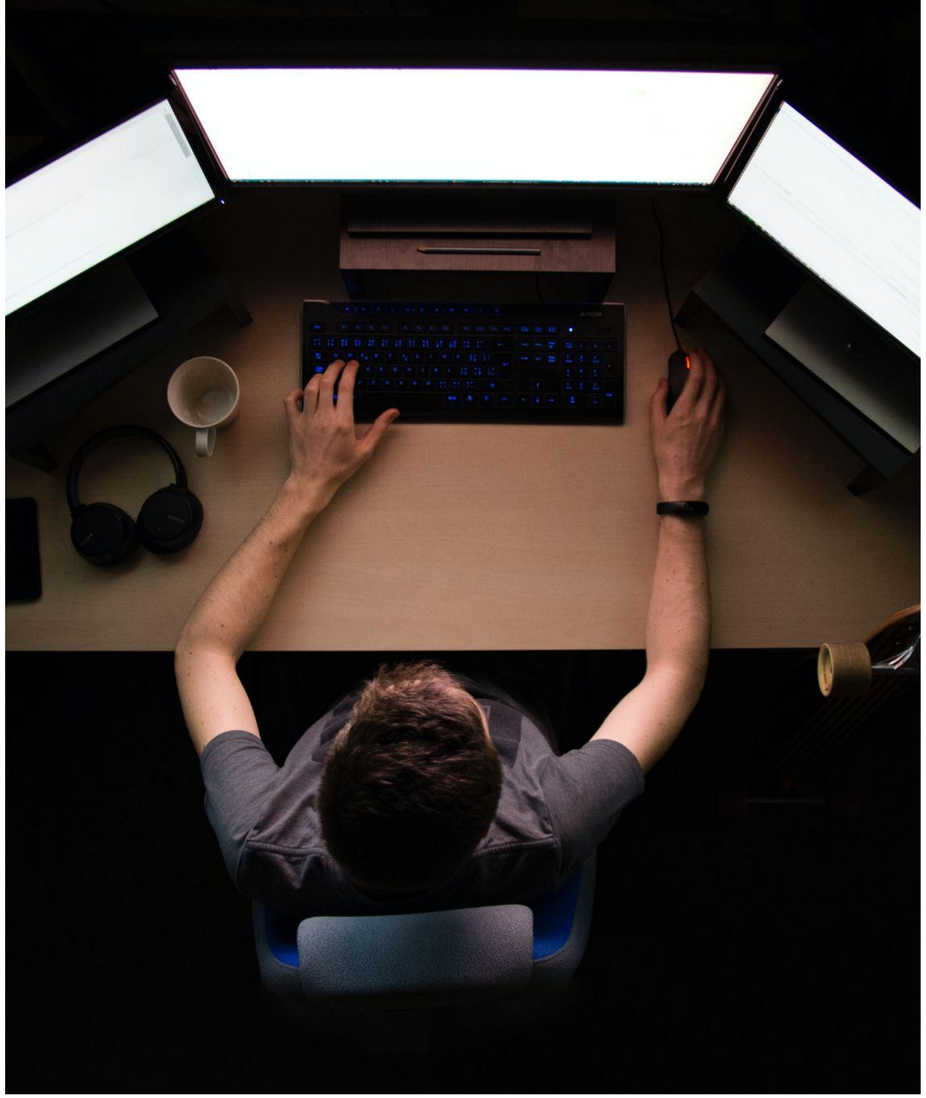
REQ. 6: DEVELOP SECURELY **OVERVIEW**

6.4 Public-facing web applications are protected against attacks

- Basically, since public-facing web apps are more vulnerable, threats and vulnerabilities must be analyzed on an ongoing basis. Also, there must be automated mechanisms to protect from attacks, and mechanisms to ensure integrity and validity of payment page scripts;

This is usually tested through:

- Examining documentation and logs on vulnerability analysis tools, payment page script integrity tools, etc;
- Verifying website configuration settings and audit logs, as well as interviewing people to ensure defenses are in order;



REQ. 6: DEVELOP SECURELY **OVERVIEW**

6.5 Changes to all system components are managed securely

- Basically, no “rogue” updates. Updates must go through an established procedure, including pre-analysis of the security impact, obtaining approval, re-analyzing PCI DSS compliance after the update, etc;
- Pre-production and production environments are separated, as roles and credentials, to minimize risks;
- Live PANs are not used in pre-production environments;

This is usually tested through:

- Examining the policies and procedures for updates, as well as records of past updates;
- Interviewing personnel and double-checking roles to ensure pre-production/production separation;

REQ. 6: DEVELOP SECURELY **OVERVIEW**

How can we summarize, then, this requirement?

- We must develop securely. Devs must be aware of security risks, analyze vulnerabilities in their apps, and code to protect against them. This includes all types (injection attacks, data attacks, cryptography abuse, etc);
- Vulnerabilities in both in-house and OTS software must be identified and patched - either internally or externally, with priority for critical vulnerabilities;
- Public-facing web apps are more vulnerable, needing automated defense mechanisms and automated payment page script integrity/authorization mechanisms;
- All updates to software must have formal procedures, including risk analysis, approval, and PCI DSS re-checking;

REQ. 6: DEVELOP SECURELY
OVERVIEW
EXAMPLES

/01 COMMON VULNERABILITIES

Common vulnerabilities can almost be considered a checklist in terms of development and testing. Injections, buffer overflows and all others are known.

/02 PATCH TIMELINE VARIES

The timeline for the installation of critical patches is not negotiable, for other patches, it is. It does need to make sense, however, as in many other requirements.

/03 FAILURE AT ANY POINT

Not addressing security can fail at any point in the SDLC. Maybe password requirements are never raised. Maybe raised but not developed. Not tested. Or other.

REQ. 6: DEVELOP SECURELY

OVERVIEW

USE CASES

SMALL COFFEE SHOP

- The organization ensures their PoS system firmware is constantly updated, immediately ranking new published vulnerabilities and patching accordingly;
- The organization selects software vendors that follow secure SDLC practices, and that provide assurance of vulnerability testing, for their integration software;

E-COMMERCE STORE

- The organization adopts the secure OWASP standards for secure coding, to prevent common web app vulnerabilities such as SQL injection or XSS;
- The organization frequently catalogs all new in-house and 3rd-party software vulnerabilities, patching critical ones as soon as possible;

HOTEL CHAIN

- The organization has a standardized process for deploying and updating software across all hotel locations, ensuring consistency and security, with reassessment of PCI DSS compliance for major changes;
- All applications are inventoried, and appropriate patching is performed;

PAYMENT PROCESSOR

- The organization has a comprehensive SDLC that integrates security at every stage, including threat modeling, code reviews, security testing and more;
- The organization uses both static and dynamic app security testing tools to detect and remediate vulnerabilities during development and testing;

REQ. 6: DEVELOP SECURELY
OVERVIEW
KEY TAKEAWAYS

/01 DEVELOP SECURELY

This requirement is about both securing third-party applications, but also your own developed ones, isolating them from vulnerabilities and patching them

/02 6.1 PROCESSES DEFINED

As usual, processes regarding this Req. must be accurate and enforced, as must be the documentation of roles and responsibilities of people involved.

/03 6.2 DEVELOPING SECURELY

Devs must be aware of vulnerabilities and threats when developing, they must protect against them, and they must be trained to do so.

/04 6.3 PATCHING VULNERS.

Vulnerabilities in both in-house and purchased SW must be patched, with critical ones in under 1 month and others at an “appropriate” pace.

/05 6.4 PUBLIC-FACING SW

Public-facing software must have automated, ongoing defense mechanisms against threats, as well as mechanisms to maintain integrity of payment scripts.

/04 6.5 SECURE CHANGES

All updates and changes to software must have a process, be approved and have their risks analyzed, and PCI DSS compliance must be re-tested after.